CUDA PSO Project

Dries Van de Velde

May 2024

1 Introduction

In a greening energy landscape, smartly scheduling energy consumption is becoming increasingly important. This is due to the increasing fluctuation in electricity production and usage.

To mitigate this discrepancy multiple tariffs are introduced in Flanders.
For this project, the most important one is the dynamic energy pricing. This is a hour based pricing strategy that uses day-ahead predictions to determine the price of electricity for the next day. This price is determined by the supply and demand of electricity. This means that the cost of electricity can vary greatly. However, it is important to note that dynamic pricing is not the total price of electricity. The total price the consumer will pay is the sum of multiple components. The dynamic price is only one of these components.

2 Project goal

The goal of this project is to create a Particle Swarm Optimisation (PSO) algorithm that can optimise the energy consumption of a household by scheduling devices. The goal is to minimise the total electricity cost. The cost is calculated by summing the product of the dynamic pricing and the energy consumption of the household. The PSO algorithm will be implemented both in C and CUDA. These implementations will be compared to each other to determine the speedup which can be achieved by using CUDA if any.

3 PSO algorithm

A PSO algorithm is a population-based optimisation algorithm. It is inspired by the social behaviour of birds. In a PSO algorithm, a population of particles is used to search for the optimal solution. Each particle represents a possible solution to the problem. The particles move through the search domain 2 and update their position based on their own best position and the best position of the swarm. This is done by using the following formulas:

$$v_i = w \cdot v_i + c_1 \cdot r_1 \cdot (p_{bestLocal)} - p_i) + c_2 \cdot r_2 \cdot (p_{bestGlobal} - p_i) \tag{1}$$

$$x_i = x_i + v_i \tag{2}$$

Where v_i is the velocity of particle i which determines the direction and speed of the particle. w is the inertia weight, which determines the influence of the previous velocity on the new velocity. c_1 and c_2 are the cognitive and social parameters, which determine the influence of the best position of the particle and the best position of the swarm on the new velocity. r_1 and r_2 are random numbers between 0 and 1, which are used to introduce randomness in the algorithm. $p_{bestLocal}$ is the best position of the particle, $p_{bestGlobal}$ is the best position of the swarm, p_i is the position of particle i.

Each of these so-called particles in the swarm represents a possible solution to the problem. All of these particles are independently evaluated and updated. This independence makes the PSO algorithm a good candidate for parallelisation.

4 Implementation

The PSO algorithm has been implemented in both C and CUDA. The algorithm uses a price array which contains all the dynamic prices for the day. For the devices which have to be matched to the dynamic prices, device arrays are used. These device arrays contain the energy consumption of the device for each time slot. When the evaluation algorithm has its data available it can start the optimisation process where the prices are calculated for different offsets. This evaluation cost is then fed back to the PSO to decide how good the solution is.

These offsets are the values used for the particles, when 2 devices are used, every particle will exist out of a length 2 array. Here, the initial values of the particle on initialisation are very important, the min and max values of these values should be set to just cover the search domain. In this case, this is 0..(23-deviceLength), as there are 24 time slots in a day. When the bounds are not properly set, there will be a lot of exploration outside of the valid solution domain, which will overall result in a less efficient algorithm.

5 Results

When comparing the C and CUDA implementations, the CUDA implementation is significantly faster for particle sizes starting at 100 as can be seen in figure 1. All the timings in the graph below are run with 1000 particle iterations, this means that every particle will be updated 1000 times.

The source code for both the c and CUDA algorithms can be found on GitHub.

References

[1] Towards Data Science. (n.d.). *Particle Swarm Optimization - Visually Explained*. Available at: https://towardsdatascience.com/particle-swarm-optimization-visually-explained-46289eeb2e14

[2] Lu, K. (2021). *CUDA-PSO*. GitHub repository. Available at: https://github.com/kailin-lu/CUDA-PSO

6 Figures

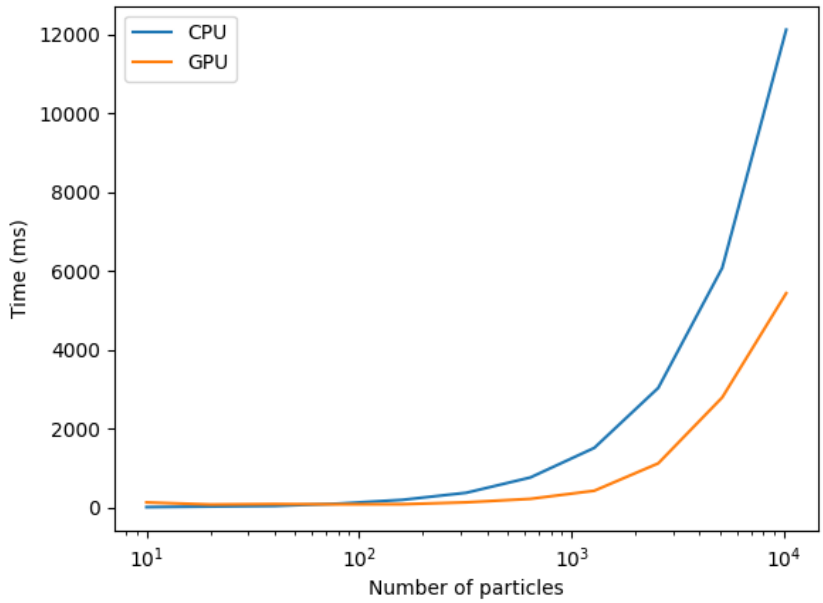


Figure 1: Execution times and number of particles with 1000 particle iterations (log scale)

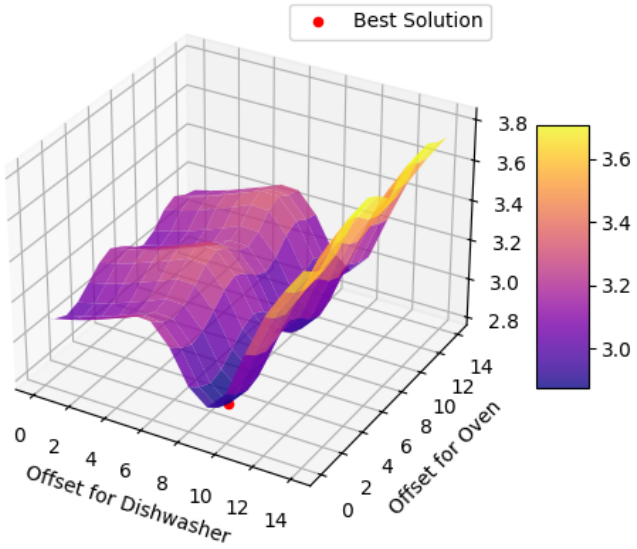


Figure 2: Search space for minimising cost when planning 2 devices with array length 5